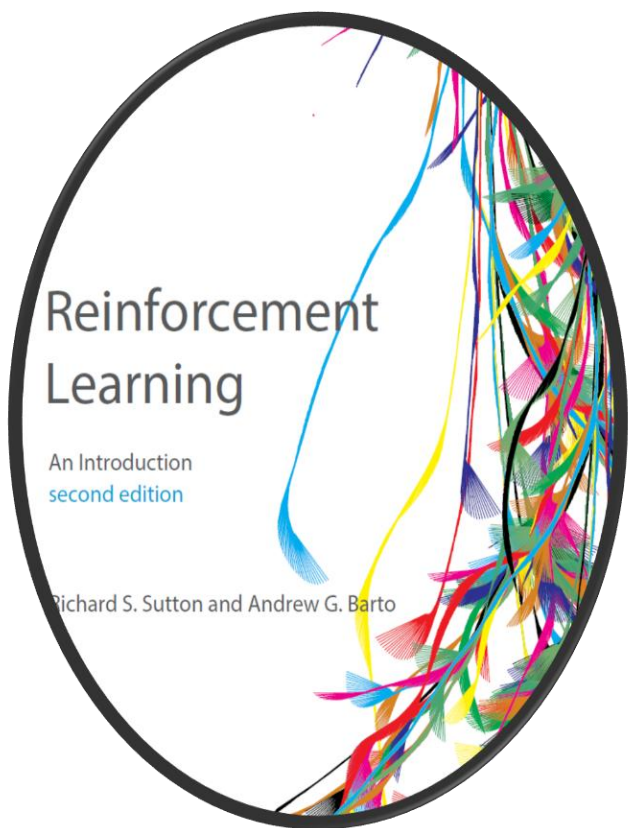




第五章：蒙特卡洛方法 (Monte Carlo Methods)

何新卫

华中农业大学信息学院

回顾：估计动作或状态价值

□ 环境动态已知（即四参数 p 已知），使用动态规划（DP）

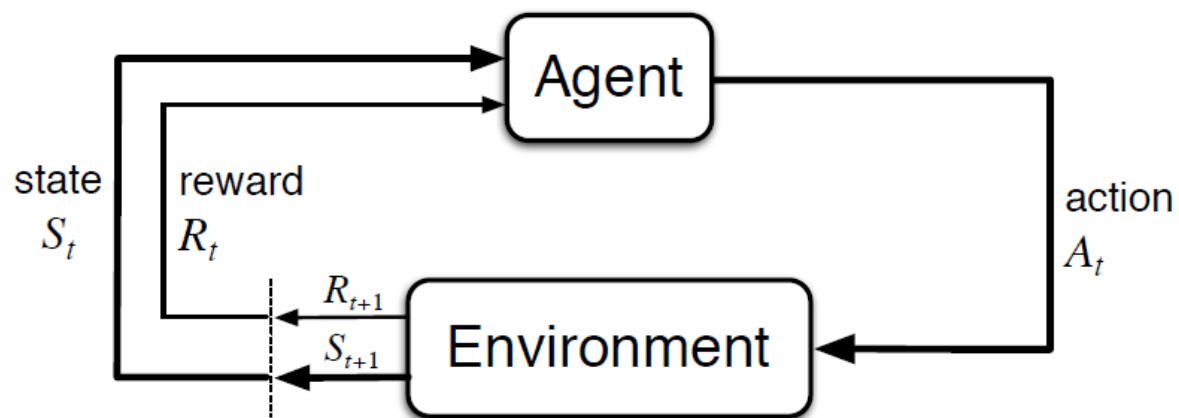
● 策略迭代：1) 策略评估 $\underline{v_{k+1}(s)} = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma \underline{v_k(s')}]$ ；2) 策略改进 $\underline{q_\pi(s,a)} = \sum_{s',r} p(s',r|s,a) [r + \gamma \underline{v_\pi(s')}]$ ；交替进行。

这种借助老的估值来更新新估值，获取到更准确估值的思想。常称为自举（Bootstrapping）

● 价值迭代： $\underline{v_{k+1}(s)} = \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma \underline{v_k(s')}]$ for all $s \in \mathcal{S}$

□ 环境动态未知（即 p 未知），无法使用动态规划

● 蒙特卡洛方法：利用交互的经验数据，估计策略下的回报（即价值）



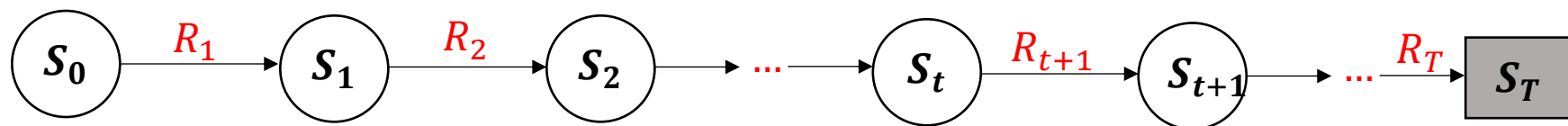
□ 智能体以某个策略 π 和MDP环境交互过程如下序列（或者轨迹（trajectory））

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, \dots$$

如何利用评估 $v_\pi(s)$, for all $s \in \mathcal{S}$

注意：每种策略都对应一个价值表 $v_\pi(s)$

回顾：回报 (Return)



□智能体与环境交互，有如下几种情形

□**回合制**：交互过程分成很子序列，这些子序列有被称为回合(**episodes**)，每个子序列有终止状态。比如，玩了一把王者荣耀，玩了一局超级玛丽，下了一局围棋等等。这些任务也被成为回合制任务（**episodic tasks**），回合制有个特殊状态叫做终止状态，即终止与于 T ，注意 T 也是一个随机变量，不同回合， T 也不一样。

□**持续性任务**：任务没有结束，即 $T = \infty$

□ T ：被称为范围(**Horizon**)

折扣回报 (Return) : 回顾

□智能体在 t 时刻，其目标是最大化期望汇报

$$G_t \triangleq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{H-1} \gamma^k R_{t+k+1}$$

□折扣系数 (discount rate) : $0 \leq \gamma \leq 1$ 超参数，对未来的奖励打折扣

✓ $\gamma=0$: 只关注即时奖励

✓ $\gamma=1$: 关注长期的累计奖励

✓ $0 < \gamma < 1$: γ 越大，越关注长期奖励，接近0，关注短期奖励

G_t 和 G_{t+1} 计算回报的关系: 回顾

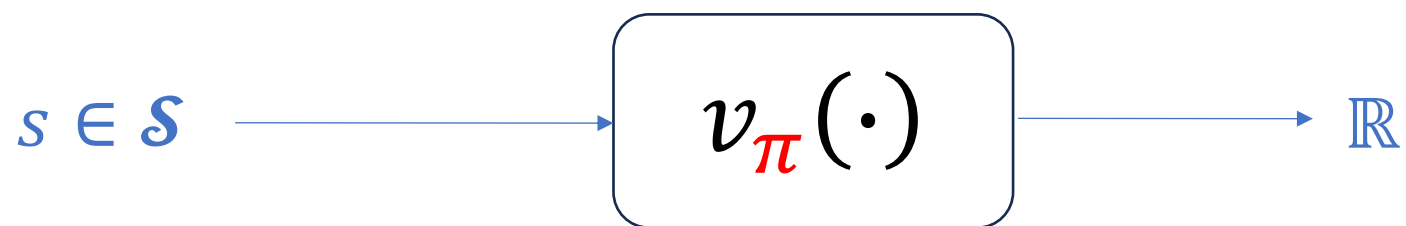
$$\begin{aligned} G_t &\triangleq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \cdots \\ &= R_{t+1} + \gamma (R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4} + \cdots) \\ &= R_{t+1} + \gamma G_{t+1} \end{aligned}$$

□ 上述公式对于 $t < H$ 均成立。

□ 对于 $t = H$, 我们定义 $G_H = 0$, 上述公式也满足。

MC预测 (Monte Carlo Prediction)

□ **MC预测问题**：未知环境，估算智能体遵循策略 π 的状态价值函数 $v_{\pi}(s)$



- **观测数据获取**：智能体（ π ）和环境交互得到的多个轨迹（episode）
 - 每个 episode中出现了一次状态 s ，称对状态 s 访问（visit）了一次。状态 s 可以在同一个episode被访问多次。
- **两类估算方法**
 - **first-visit MC 方法**：在估计回报时，只考虑状态 s 首次被访问的回报。
 - **every-visit MC 方法**：对状态 s 所有visit 均估算回报，然后平均。
- 基于**大数定理**，**first-visit**和**every-visit** MC随着状态 s 访问次数趋于无穷，则估值将逼近真实 $v_{\pi}(s)$

first-visit MC 方法

First-visit MC prediction, for estimating $V \approx v_\pi$

Input: a policy π to be evaluated

待评估策略

Initialize:

$V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$

$Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

初始化价值函数和近似函数

Loop forever (for each episode):

Generate an episode following π : $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_{T-1}, A_{T-1}, R_T$

遵循策略，交互生成
观测数据

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless S_t appears in $S_0, S_1, \dots, S_{t-1}$:

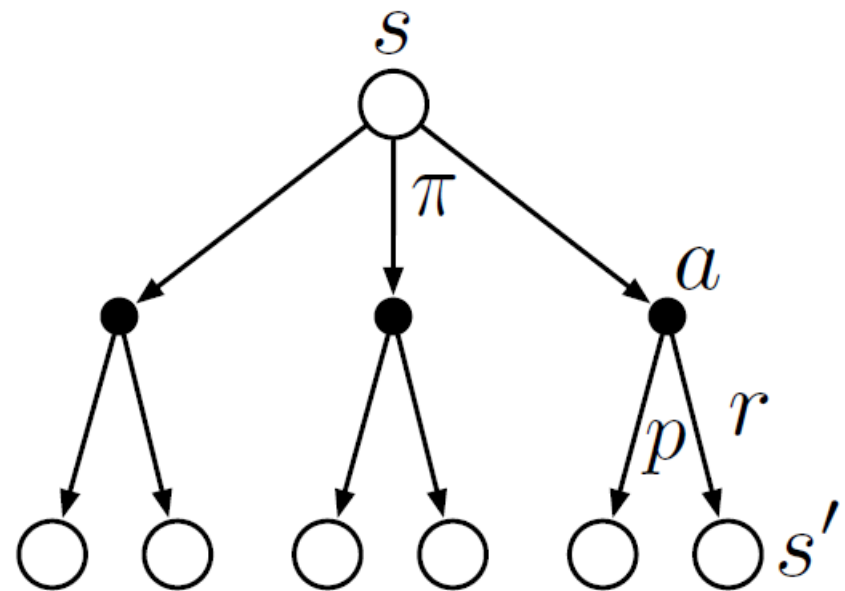
Append G to $Returns(S_t)$

$V(S_t) \leftarrow \text{average}(Returns(S_t))$

估算本条episode对应状态的回报【从后往前计算】

即如果该具体状态在该时刻前面没有出现过，表明这是首次出现，则执行后面语句：将估算回报加入近似评估函数，并平均，作为该状态价值估计

MC和DP的区别：以备份图为例



Backup diagram for v_π

DP: 一步转移，考虑所有可能转移，状态价值非独立，用到了自举（bootstrap）



直至episode终止。不同状态之间估値是**独立的**，即没有bootstrap

关于DP和MC的价值估算方法对比，以下说法正确的是

- ☒ A 不管未知环境还是已知环境，MC均适用
- ☒ B DP仅适用已知环境
- ☐ C MC仅适用未知环境。
- ☒ D 当对某一个状态价值比较感兴趣的时候，MC更加简便

提交

MC优点

- **普适性：** 未知、已知环境均适用： 均可通过采样生成观测序列（episode）， 完成价值估算， 而DP只适用于已知环境
- **灵活性：** 相比DP， 更加灵活。当只关心某个状态或者某几个状态的价值
 - 仅从当前关心的状态出发， 采样大量的episodes
 - 仅对这些状态计算回报并平均
 - 不用计算其它不关心的状态

MC估计动作价值

- 模型未知：仅估算出状态价值不足以帮我们确定最优策略。必须明确估算出每个动作的价值，才能确定最优策略。
- 模型已知：借助状态价值也可帮我们决定策略，即只需要向前选择一次动作，并找到能够带来最大回报的动作即可（DP）。
- 因此MC方法的主要目标就是估算最优动作价值，即 q_*
- 策略评估主要目的是估算 $q_\pi(s, a)$ ，即从状态 s 出发，执行动作 a ，而后一致遵循策略 π 的期望回报。
- MC方法估算状态-动作对价值 $q_\pi(s, a)$ 和估算状态价值 $v_\pi(s)$ 过程完全一样，只不过将状态换成了状态-动作对而已。

MC估计动作价值

□ 两类方法

- **Every-visit MC**: 对所有出现(s, a)计算回报, 取均值, 作为其价值估值
- **First-visit MC**: 只对每个episode第一次出现的 (s, a) 计算回报作为其价值估值。

□ 唯一难题:

- 很多状态-动作对**可能永远不会被访问到**。比如, 如果策略 π 是一个确定的策略, 那么遵循 π , 我们将只能观测到当前状态下执行某一确定性动作的回报, 也就是说我们没法积累经验数据改进其它剩余动作的价值。
- 这意味着我们没法通过学习其它可选动作价值来帮助我们选择当前状态下更优的动作了
- **探索-利用。**

MC估计动作价值

- 持续探索：为了能正确估计给定状态下所有可能动作的价值，方便比较，优中选优，我们必须持续探索。
- 探索性开始（ **exploring starts** ）：一个比较好的解决办法就是在episode的开始时保证随机性，即不同的状态-动作对均以非0的概率被选中作为起始状态-动作。从而确保所有的状态-动作对在无穷的episodes下被无穷次的访问
- 在开始仅考虑随机策略，在开始状态，保证以非零概率选择其它所有可能的动作。

蒙特卡洛估计动作价值

□ 利用经验，评估 $q_{\pi}(s, a)$ (即每个状态-动作对的价值)， for all $s \in \mathcal{S}$

Monte Carlo ES (Exploring Starts), for estimating $\pi \approx \pi_*$

Initialize:

$\pi(s) \in \mathcal{A}(s)$ (arbitrarily), for all $s \in \mathcal{S}$

$Q(s, a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$

$Returns(s, a) \leftarrow$ empty list, for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$

Loop forever (for each episode):

Choose $S_0 \in \mathcal{S}, A_0 \in \mathcal{A}(S_0)$ randomly such that all pairs have probability > 0

Generate an episode from S_0, A_0 , following π : $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1, \dots, S_{t-1}, A_{t-1}$:

Append G to $Returns(S_t, A_t)$

$Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$

$\pi(S_t) \leftarrow \operatorname{argmax}_a Q(S_t, a)$

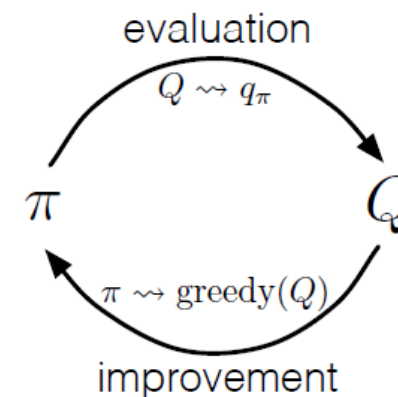
随机开始
采样轨迹

回报估值
评估每个状态动作对的价值

MC 控制问题

- 完成策略评估后，接下来需要基于评估结果找最优策略。
- 整体思路类似DP。即**策略评估**和**策略提升**交替进行

$$\pi_0 \xrightarrow{E} q_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} q_{\pi_1} \xrightarrow{I} \pi_2 \rightarrow \cdots \rightarrow \pi_* \xrightarrow{E} q_{\pi_*}$$



- **策略改进**：基于当前价值函数贪心改进。
- **贪心策略**：基于估算的动作价值函数 q ，我们能直接构建贪心策略
- 对于动作价值函数 q ，贪心策略就是：对任意状态 $s \in \mathcal{S}$ ，选择具有最大的动作价值的动作，即

$$\pi(s) = \underset{a}{\operatorname{argmax}} q(s, a)$$

MC 控制问题

- 策略改进：可通过基于 q_{π_k} 构建一个新的贪心策略 π_{k+1}
- 上章的策略提升理论也适用于 π_k 和 π_{k+1} ，这是因为对于任意 $s \in \mathcal{S}$ ，我们有

$$\begin{aligned} q_{\pi_k}(s, \pi_{k+1}(s)) &= q_{\pi_k}(s, \arg \max_a q_{\pi_k}(s, a)) \\ &= \max_a q_{\pi_k}(s, a) \\ &\geq v_{\pi_k}(s). \end{aligned}$$

$$\sum_{i=1}^n w_i x_i < x_{\max} \sum_{i=1}^n w_i$$

$$\frac{\sum_{i=1}^n w_i x_i}{\sum_{i=1}^n w_i} < x_{\max}$$

蒙特卡洛控制：摒弃探索性开始 (Exploring Starts)

- 探索性开始，效率太低
- 更通用性的策略就是：智能体将在后续交互中以**非零概率持续选择所有可能的动作**，有两种方法可以保证。

摒弃探索性开始的MC (Monte Carlo Control without Exploring Starts)

□ 软性策略 $\pi(a|s)$

● **for all** $s \in \mathcal{S}$, $a \in \mathcal{A}$, $\pi(a|s) > 0$

● 逐渐收敛到确定性的最优策略上

□ ϵ -贪婪策略: ϵ -贪婪策略是 ϵ -软策略中的一种特殊例子。

● $1 - \epsilon$ 的概率选择具有最大估计动作值的动作, 但以概率 ϵ 选择一个随机动作。

● 在 ϵ -软策略中, ϵ -贪婪策略在某种意义上是最接近贪婪策略的策略。

On-policy first-visit MC control (for ε -soft policies), estimates $\pi \approx \pi_*$

Algorithm parameter: small $\varepsilon > 0$

Initialize:

$\pi \leftarrow$ an arbitrary ε -soft policy

$Q(s, a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

$Returns(s, a) \leftarrow$ empty list, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

Repeat forever (for each episode):

Generate an episode following π : $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1, \dots, S_{t-1}, A_{t-1}$:

Append G to $Returns(S_t, A_t)$

$Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$

$A^* \leftarrow \arg\max_a Q(S_t, a)$ (with ties broken arbitrarily)

For all $a \in \mathcal{A}(S_t)$:

$$\pi(a|S_t) \leftarrow \begin{cases} 1 - \varepsilon + \varepsilon/|\mathcal{A}(S_t)| & \text{if } a = A^* \\ \varepsilon/|\mathcal{A}(S_t)| & \text{if } a \neq A^* \end{cases}$$

Monte Carlo Control 正是将蒙特卡洛预测与策略提升结合起来

对任何一个 ε -软策略 π ， 基于 q_π 得到 ε -贪心策略能保证优于或者不差于 π

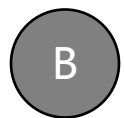
$$\begin{aligned}
 q_\pi(s, \pi'(s)) &= \sum_a \pi'(a|s) q_\pi(s, a) \\
 &= \frac{\varepsilon}{|\mathcal{A}(s)|} \sum_a q_\pi(s, a) + (1 - \varepsilon) \max_a q_\pi(s, a) \\
 &= \frac{\varepsilon}{|\mathcal{A}(s)|} \sum_a q_\pi(s, a) + (1 - \varepsilon) \sum_a \frac{\pi(a|s) - \frac{\varepsilon}{|\mathcal{A}(s)|}}{1 - \varepsilon} \max_a q_\pi(s, a) \\
 &\geq \frac{\varepsilon}{|\mathcal{A}(s)|} \sum_a q_\pi(s, a) + (1 - \varepsilon) \sum_a \frac{\pi(a|s) - \frac{\varepsilon}{|\mathcal{A}(s)|}}{1 - \varepsilon} q_\pi(s, a) \quad \boxed{\frac{\sum_{i=1}^n w_i x_i}{\sum_{i=1}^n w_i} < x_{\max}} \\
 &= \frac{\varepsilon}{|\mathcal{A}(s)|} \sum_a q_\pi(s, a) - \frac{\varepsilon}{|\mathcal{A}(s)|} \sum_a q_\pi(s, a) + \sum_a \pi(a|s) q_\pi(s, a) \\
 &= v_\pi(s)
 \end{aligned}$$

因此， 上述满足策略提升理论， 即 $v_{\pi'}(s) > v_\pi(s)$, **for all** $s \in \mathcal{S}$

通过上述的方法收敛后学习到的策略是



接近最优策略的 ϵ -贪婪策略



确定性最优策略

提交

动机

- **困境:** 想学最优确定策略，却必须通过非最优策略去探索。
- 能否使用一个**随机策略去探索环境**，其数据**用于学习确定的最优策略**呢？



异策略学习 (Off-Policy Learning)

□ 异策略 (Off-policy) : 使用两种策略

- 目标策略 (target policy) : 被学习的并成为最优策略的策略
- 行为策略 (behavior policy): 更具探索性, 用于生成轨迹数据的策略。



两种学习方法对比

	同策略 (on-policy)	异策略(off-policy)
行为策略	当前 ϵ -贪婪策略	通常是更探索性的策略（如随机策略）
目标策略	同一个 ϵ -贪婪策略	最终的最优贪婪策略
学习的数据来源	自己产生的经验	别人的经验（可能来自不同策略）
能否学习最优 确定性策略	不能，只能学到近优的 ϵ -贪婪策略	可以学到最优确定性策略

异策略基础理论：重要性采样

- 根据目标策略和行为策略下轨迹相对概率，对异策略的回报加权处理。
- 从初始状态 S_t 出发，遵循策略 π ，生成状态-动作轨迹 $A_t, S_{t+1}, A_{t+1}, \dots, S_T$ 的概率(MDP)如下：

$$\begin{aligned}\Pr(A_t, S_{t+1}, A_{t+1}, \dots, S_T | S_t, A_{t:T-1} \sim \pi) \\&= \pi(A_t | S_t) p(S_{t+1} | S_t, A_t) \pi(A_{t+1} | S_{t+1}) \dots p(S_T | S_{T-1}, A_{T-1}) \\&= \prod_{k=t}^{T-1} \pi(A_k | S_k) p(S_{k+1} | S_k, A_k)\end{aligned}$$

- 重要性系数：目标策略(π)和行为策略(b)对同一轨迹的相对概率为：

$$\rho_{t:T-1} = \frac{\prod_{k=t}^{T-1} \pi(A_k | S_k) p(S_{k+1} | S_k, A_k)}{\prod_{k=t}^{T-1} b(A_k | S_k) p(S_{k+1} | S_k, A_k)} = \prod_{k=t}^{T-1} \frac{\pi(A_k | S_k)}{b(A_k | S_k)}$$

- 重要性抽样比率最终只取决于这两种策略和具体轨迹，与环境无关。

目标和行为策略的状态价值函数关系

□ 目标：估计目标策略下状态的期望回报（价值），即 $v_\pi(s)$ ？

□ 条件：基于行为策略数据估算出来的回报 G_t ，期望值如下：

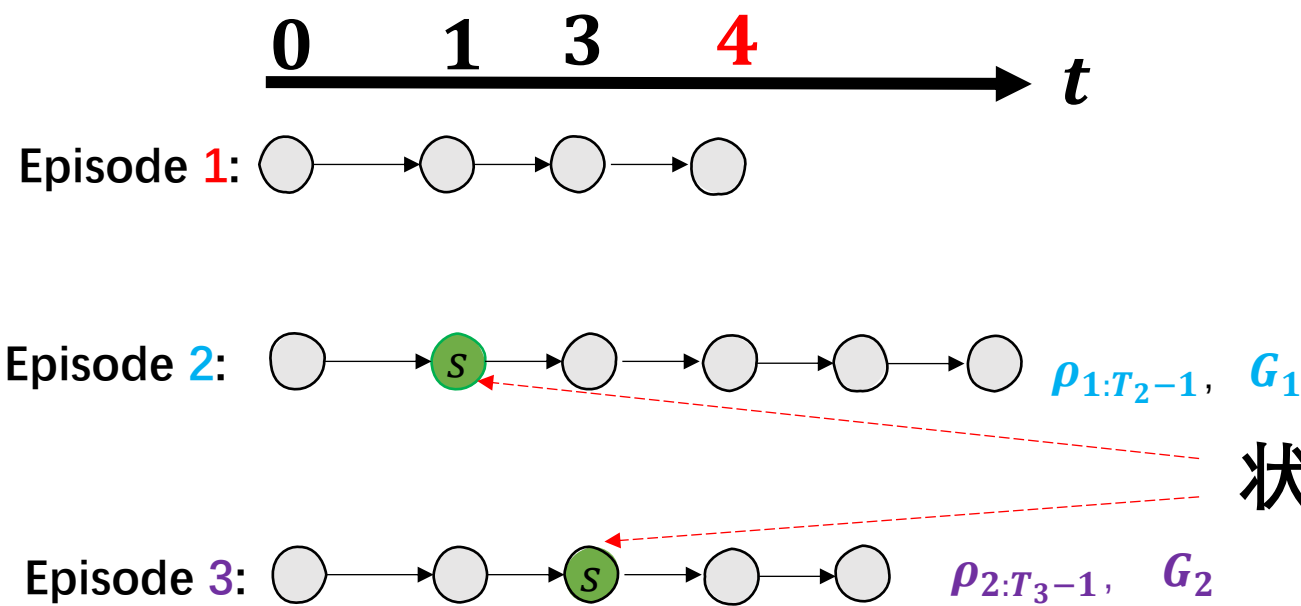
$$v_b(s) = \mathbb{E}[G_t | S_t = s]$$

□ 基于重要性采样比率 $\rho_{t:T-1}$ ，则回报 $v_\pi(s)$ ：

$$v_\pi(s) = \mathbb{E}[\underline{\rho_{t:T-1}} G_t | S_t = s]$$

普通重要性采样：以First-visit MC为例

- 目标：估计 $v_{\pi}(s)$
- 条件：遵循策略 b 的一批观察到的回合中平均返回值。



状态 s 在相应 episodes 被首次访问

$$V(s) \triangleq \frac{\textcolor{blue}{\rho_{1:T_2-1}} \textcolor{blue}{G_1} + \textcolor{purple}{\rho_{2:T_3-1}} \textcolor{purple}{G_2}}{\textcolor{red}{2}} \begin{matrix} \text{回报总和} \\ \text{状态 } s \text{ 被首次访问的次数} \end{matrix}$$

根据大数定律，只要我们得到足够多的轨迹，就可以趋近这个策略对应的价值函数。当 $N(s) \rightarrow \infty$ 时， $V(s) \rightarrow v_{\pi}(s)$ 。

目标策略价值估计一般公式 (MC first)

□ 约定在episode边界出累加时间步。如第一个episode在100步终止，则下一episode第一步为101步。

- 状态 s 被访问的所有时间步骤的集合，记为 $\mathcal{T}(s)$;
- $\mathcal{T}(s)$ 只包括其回合内首次访问 s 的时间步。

□ $T(t)$ 表示时间 t 之后第一次终止的时间，并且 G_t 表示从 t 到 $T(t)$ 结束的回报

□ 那么， $\{G_t\}_{t \in \mathcal{T}(s)}$ 是状态 s 的回报估值集合，而 $\{\rho_{t:T(t)-1}\}_{t \in \mathcal{T}(s)}$ 是相应比例。

为估计 $v_\pi(s)$ ，我们简单通过这些比率对回报进行缩放并平均：

$$V(s) \doteq \frac{\sum_{t \in \mathcal{T}(s)} \rho_{t:T(t)-1} G_t}{|\mathcal{T}(s)|}$$

加权重要性采样

加权重要性采样，它使用加权平均，定义为：

$$V(s) \doteq \frac{\sum_{t \in \mathcal{T}(s)} \rho_{t:T(t)-1} G_t}{\sum_{t \in \mathcal{T}(s)} \rho_{t:T(t)-1}}$$

□特别的，考虑在观察到状态s的单次episode回报后。

- 加权平均估计，单次回报的比率 $\rho_{t:T(t)-1}$ 在分子和分母中相互抵消。但其期望是 $v_b(s)$ 而不是 $v_\pi(s)$ ，
- 在统计意义上，它是有偏的。
- 假设比率为十，表示所观察到的轨迹在目标策略下可能性比行为策略下更有可能出现十倍。即使这一episode的轨迹被认为非常符合目标策略，它与观察到的回报相差很远，具有较高的variance。

MC预测方法的增量实现 (Incremental Implementation)

□ 假设一个回报序列，记 $G_1, G_2, G_3, \dots, G_{n-1}$ ，设每一个回报元素均起始于同一个状态 s ，对应一个权重变量 $W_i = \rho_{t_i:T(t_i)-1}$ ，需估计如下价值函数：

$$V_n(s) \triangleq \frac{\sum_{k=1}^{n-1} W_k G_k}{\sum_{k=1}^{n-1} W_k}, n \geq 2$$

□ 增量更新公式如下：

$$V_{n+1}(s) \triangleq V_n(s) + \frac{W_n}{C_n} (G_n - V_n(s)), n \geq 1$$

$$C_{n+1} \triangleq C_n + W_{n+1}$$

其中， $C_0 \triangleq 0$

$$V_{n+1} = \frac{W_n G_n + \sum_{k=1}^{n-1} W_k G_k}{\sum_{k=1}^n W_k}$$

$$\sum_{k=1}^{n-1} W_k G_k = V_n * \sum_{k=1}^{n-1} W_k$$

$$V_{n+1} = \frac{W_n G_n + V_n * \sum_{k=1}^{n-1} W_k}{\sum_{k=1}^n W_k}$$

$$V_{n+1} = \frac{W_n G_n + V_n * \sum_{k=1}^{n-1} W_k + V_n W_n - V_n W_n}{\sum_{k=1}^n W_k}$$

$$V_{n+1} = \frac{W_n G_n + V_n (W_n + \sum_{k=1}^{n-1} W_k) - V_n W_n}{\sum_{k=1}^n W_k}$$

$$V_{n+1} = V_n + \frac{W_n G_n - W_n V_n}{\sum_{k=1}^n W_k}$$

Off-policy MC prediction (policy evaluation) for estimating $Q \approx q_\pi$

Input: an arbitrary target policy π

Initialize, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$:

$Q(s, a) \in \mathbb{R}$ (arbitrarily)

$C(s, a) \leftarrow 0$

Loop forever (for each episode):

$b \leftarrow$ any policy with coverage of π

Generate an episode following b : $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

$W \leftarrow 1$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$, while $W \neq 0$:

$G \leftarrow \gamma G + R_{t+1}$

$C(S_t, A_t) \leftarrow C(S_t, A_t) + W$

$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)} [G - Q(S_t, A_t)]$

$W \leftarrow W \frac{\pi(A_t|S_t)}{b(A_t|S_t)}$

- 针对异策略情况下的加权重要性采样
- 当目标和行为策略相同（即 $\pi=b$ ），即同策略（此时 W 始终为1），同样适用
- $Q(s, a)$ 收敛于 $q_\pi(s, a)$ （**for all** $s \in \mathcal{S}$, $a \in \mathcal{A}$ ），动作选择遵循策略 b

异策略 MC控制 (Off-policy Monte Carlo Control)

□ 智能体两种策略

- 行为策略：生成轨迹的策略被称为行为策略 (behavior policy)
- 目标策略：评估和改进的策略。

□ 分离的优势

- 目标策略可以是确定性的（例如，贪婪策略），而行为策略可以继续采样所有可能的动作。
- 为了探索所有可能性，我们要求行为策略它以非零概率选择所有状态中的所有动作（又被称为 ϵ -soft）。

□ 目标策略 (π)：是关于 Q 的贪心策略。

□ 行为策略 (b)：是 ϵ -soft的策略，策略 π 将收敛于 π_* ，即使所有的动作都来自行为策略 b

Off-policy MC control, for estimating $\pi \approx \pi_*$

Initialize, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$:

$Q(s, a) \in \mathbb{R}$ (arbitrarily)

$C(s, a) \leftarrow 0$

$\pi(s) \leftarrow \operatorname{argmax}_a Q(s, a)$ (with ties broken consistently)

Loop forever (for each episode):

$b \leftarrow$ any soft policy

Generate an episode using b : $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

$W \leftarrow 1$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

$C(S_t, A_t) \leftarrow C(S_t, A_t) + W$

$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)} [G - Q(S_t, A_t)]$

$\pi(S_t) \leftarrow \operatorname{argmax}_a Q(S_t, a)$ (with ties broken consistently)

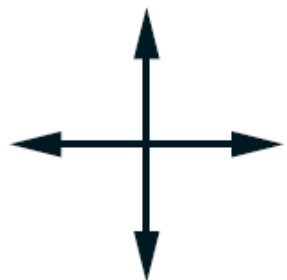
If $A_t \neq \pi(S_t)$ then exit inner Loop (proceed to next episode)

$W \leftarrow W \frac{1}{b(A_t|S_t)}$

当行为策略的动作偏离目标策略时，后续的重要性采样权重为零，学习无法进行，因此提前终止该 episode 的处理，节约计算资源。

案例和实现分析

环境介绍:



actions

	1	2	3
4	5	6	7
8	9	10	11
12	13	14	

$R_t = -1$
on all transitions

$$\mathcal{S} = \{1, 2, \dots, 14\} \quad \mathcal{A} = \{\text{up, down, right, left}\}$$

□ 4x4的格子

□ 灰色为终止状态，到达终止态，奖励为5，转移到其它状态奖励-1

环境实现

```
8 class GridWorldEnv(Env):
9     """Custom Environment that follows gym interface"""
10
11     def __init__(self):
12         self.action_space = Discrete(4)
13         self.observation_space = Tuple((Discrete(4), Discrete(4)))
14         # Initialize the grid
15         self.grid = np.zeros([4,4])
16         self.grid[0][0] = 1
17         self.grid[3][3] = 1
18         self.max_steps = 50
19         self.step_count = 0
```

动作空间和状态空间

定义环境

环境支持最多行走50步，超过则强制退出

环境实现 (Cont.)

```
21 def step(self, action):
22     new_pos = None
23     if action == 0: # up
24         new_pos = (self.pos[0] - 1, self.pos[1])
25     elif action == 1: # right
26         new_pos = (self.pos[0], self.pos[1] + 1)
27     elif action == 2: # down
28         new_pos = (self.pos[0] + 1, self.pos[1])
29     elif action == 3: # left
30         new_pos = (self.pos[0], self.pos[1] - 1)
31     if self._onGrid(new_pos):
32         self.pos = new_pos
33     done, reward = False, -1
34     if self._inTerminalPos():
35         done = True
36         reward = 5
37     if self.step_count == self.max_steps:
38         done = True
39     self.step_count += 1
40     return self.pos, reward, done, {}
41
```

环境接收动作

计算下一个状态

是否终止、获取奖励

下一个状态、奖励、是否结束，其它

环境实现 (Cont.)

```
43 def _onGrid(self, pos):
44     return pos[0] in range(0,4) and pos[1] in range(0,4)
45
46 def _inTerminalPos(self):
47     return self.grid[self.pos[0]][self.pos[1]] == 1
48
49 def reset(self):
50     # Reset step count
51     self.step_count = 0
52     # Start the agent in the top right corner
53     self.pos = (0,3)
54     return self.pos
55
56 # Optionally allow start position to be set
57 def setStartPos(self, pos):
58     if self._onGrid(pos):
59         self.pos = pos
60     else:
61         print("ERROR: Start position not on grid")
```

该位置是否还在Grid上面

该位置是否终止状态

初始化，进行重置其实位置位于右上角

支持设置新的起始位置

策略实现

```
5 class Policy(object):
6     def __init__(self, p_type, epsilon=0.1):
7         # d: deterministic, nd: non-deterministic, eg: epsilon-greedy
8         valid_policy_types = ['d', 'nd', 'eg']
9         assert(p_type in valid_policy_types)
10        self.p_type = p_type
11        self.policy = {}
12        self.epsilon = epsilon
13        self.initPolicyValues()
14
15    def initPolicyValues(self):
16        for x in range(4):
17            for y in range(4):
18                if self.p_type == 'nd':
19                    # 非确定性策略, 每个动作的概率相等
20                    self.policy[(x,y)] = [0.25] * 4
21                else:
22                    # 确定性策略和epsilon贪心策略, 随机选择一个动作
23                    self.policy[(x,y)] = random.randint(0,3)
```

三种策略（即状态-动作映射函数）

每个状态，设置要选择的动作

随机策略：四个动作等概率选择

确定性策略，也可以是epsilon贪心策略

策略实现(Cont.)

```
25 def getPolicyAction(self, s):
26     if self.p_type == 'd':
27         return self.policy[s]
28     elif self.p_type == 'nd':
29         return self._getNonDetPolicyAction(s)
30     elif self.p_type == 'eg':
31         return self._getEpGreedyPolicyAction(s)
32 def _getNonDetPolicyAction(self, s):
33     return np.random.choice(np.arange(0,4),p=self.policy[s])
34 def _getEpGreedyPolicyAction(self, s):
35     if random.uniform(0,1) < self.epsilon:
36         # Explore
37         return random.choice(list(range(4)))
38     else:
39         # Exploit
40         return self.policy[s]
41
42 # Update policy to choose action 'a' when in state 's'
43 def updateDetPolicyState(self, s, a):
44     self.policy[s] = a
```

输入相应状态，基于策略
选择动作

基于动作概率选择

Epislon贪心选择动作
(探索和利用的权衡)

更新策略：输入状态-动作

MC估计状态价值实现 First-visit MC

设置

```
5 env = GridWorldEnv()
6 num_episodes, gamma = 1000, 0.9
7 def firstVisitMCPred(gamma, num_episodes):
8     policy = Policy('nd')
9     V, returns = {}, {}
10    for x in range(4):
11        for y in range(4):
12            V[(x,y)], returns[(x,y)] = 0, 0
13    for _ in tqdm(range(num_episodes)):
14        trajectory = getTrajectory(env, policy)
15        G = 0 # 初始化G
16        for t in reversed(range(len(trajectory))):
17            # 从轨迹的这一状态开始
18            s_t, a_t, r_t = trajectory[t]
19            G = gamma * G + r_t
20            # 如果这是该状态第一次出现
21            if isFirstVisit(s_t):
22                returns[s_t] += G
23            V[s_t] = average(returns[s_t])
24    firstVisitMCPred(gamma, num_episodes)
```

```
61 def genEpisode(env, policy, es=False):
62     obs = env.reset()
63     done = False
64     if es: # 探索性开始
65         # 选择一个随机的初始状态和随机动作
66         x = random.randint(0,3)
67         y = random.randint(0,3)
68         a = random.randint(0,3)
69         env.setStartPos((x,y))
70         obs, reward, done, info = env.step(a)
71         trajectory, t = {}, 0
72         while not done:
73             s = obs
74             action = policy.getPolicyAction(s)
75             obs, reward, done, info = env.step(action)
76             trajectory[t] = [s, action, reward]
77             t += 1
78         return trajectory
```

轨迹,
回报,
回报,
迭代

First-visit MCES (Exploration start探索性开始)

```
27 def MCES(gamma, policy, env, es=False):
28     policy = policy
29     # 初始化Q (green)
30     Q, returns = {}
31     for x in env.reset():
32         for y in env.reset():
33             for a in env.reset():
34                 for _ in range(1000):
35                     # 生成轨迹 (blue)
36                     trajectory = []
37                     G = 0
38                     for t in range(1000):
39                         s, a, r = env.step(a)
40                         trajectory.append((s, a, r))
41                         G += r
42                         if done:
43                             break
44                     Q[(x, y)] = G
45                     returns[(x, y)] = G
46     MCES(gamma, policy, env, es=False)
```

```
61 def genEpisode(env, policy, es=False):
62     obs = env.reset()
63     done = False
64     if es: # 探索性开始 (green)
65         # 选择一个随机的初始状态和随机动作 (green)
66         x = random.randint(0,3)
67         y = random.randint(0,3)
68         a = random.randint(0,3)
69         env.setStartPos((x,y))
70         obs, reward, done, info = env.step(a)
71     trajectory, t = {}, 0
72     while not done:
73         s = obs
74         action = policy.getPolicyAction(s)
75         obs, reward, done, info = env.step(action)
76         trajectory[t] = [s, action, reward]
77         t += 1
78     return trajectory
```

初始化

生成轨迹

状态-动作回报

更新策略

On-policy first visit MC

```
49 def OnPolicyFirstVistMCControl(
50     policy = Policy('eg', epsilon),
51     # 初始化Q (动作值函数) 和
52     Q, returns,
53     for x in
54     for
55     for
56     for
57     for
58     for in
59     traj
60     G =
61     for
62
63
64
65
66
67
68 OnPolicyFirstVistMCControl(gamma, num_episodes)
```

```
61 def genEpisode(env, policy, es=False):
62     obs = env.reset()
63     done = False
64     if es: # 探索性开始
65         # 选择一个随机的初始状态和随机动作
```

```
94 def getMaxActionForState(s, Q):
95     max_value = float('-inf')
96     max_action = None
97     for (state, action), value in Q.items():
98         if state == s:
99             if value > max_value:
100                 max_action = action
101                 max_value = value
102     return max_action
```

```
returns[(s_t, a_t)].append(G)
Q[(s_t, a_t)] = average(returns[(s_t, a_t)])
policy.updateDetPolicyState(s_t, getMaxActionForState(s_t, Q))
```

epsilon-
价值和回

Off-policy every visit MC控制

```
71 # 使用增量更新的离策略MC控制实现 (使用加权重要性采样)
```

```
72 def OffPolicyEveryVistMCControl(gamma, num_episodes):
```

```
73     pi, b = Policy('d'), Policy('nd')
```

```
74     Q, C = {}, {}
```

```
75     for x in range(4):
```

```
76         for y in range(4):
```

```
77             for action in range(4):
```

```
78                 Q[((x,y), action)], C[((x,y), action)] = 0, 0
```

```
79     for _ in tqdm(range(num_episodes)):
```

```
80         trajectory = genEpisode(env, b)
```

```
81         G, W = 0, 1
```

```
82         for t in reversed(trajectory):
```

```
83             s_t, a_t, r_t_plus_1 = trajectory[t]
```

```
84             G = gamma * G + r_t_plus_1
```

```
85             C[((x,y), action)] += W
```

```
86             Q[(s_t, a_t)] += (W / C[(s_t, a_t)]) * (G - Q[(s_t, a_t)])
```

```
87             pi.updateDetPolicyState(s_t, getMaxActionForState(s_t, Q))
```

```
88             if a_t != pi.policy[s_t]:
```

```
89                 break
```

```
90                 W *= 1 / b.policy[s_t][a_t]
```

```
91 OffPolicyEveryVistMCControl(gamma, num_episodes)
```

初始化, 注意目标策略和行为策略

增量更新

如果行为策略的动作和目标策略不符合, 则直接退出, 后续状态均记录为0

总结

- Off-policy 和 on-policy
- MC first visit 和 MC every visit 区别
- MC first visit exploring start
- Off policy MC 的理论保证
- MC 增量实现
- 代码实现